

# Selezione Dinamica della Dimensione del Campione

in Metodi di Ottimizzazione per il Machine Learning

---

La tesi presenta la selezione dinamica della dimensione del campione negli algoritmi di ottimizzazione per il machine learning su larga scala, fondandosi sul lavoro di Byrd, Chin, Nocedal e Wu (2012). L'idea centrale: invece di fissare a priori la dimensione del batch, si decide durante l'algoritmo se e quando aumentarla, guidandosi da stime statistiche della varianza del gradiente stocastico. Il lavoro offre un'analisi teorica completa della convergenza, il miglioramento di alcuni risultati mediante disuguaglianze più strette e un'implementazione Python dei quattro algoritmi principali: Dynamic GD, Newton-CG, estensione  $L_1$  e BB-CCV, con un'applicazione web interattiva per la sperimentazione.

## Contesto: ottimizzazione su larga scala

- Obiettivo: minimizzare la perdita empirica

$$J(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i), \quad w \in \mathbb{R}^m$$

con  $N \approx 10^6$ – $10^9$  esempi e  $m$  grande: il gradiente completo  $\nabla J(w)$  costa  $O(Nm)$  per iterazione, proibitivo.

- Mini-batch  $\mathcal{S}_k$  di dimensione  $n_k$ : stima  $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ , costo  $O(n_k m)$ . Se  $n_k \ll N$  le iterazioni sono economiche.
- Compromesso classico (Bottou–Bousquet, 2008): batch piccolo = stima rumorosa; batch grande = stima accurata ma costo proporzionale alla dimensione.
- Nei metodi classici  $n_k$  è **fissata a priori**; questa tesi analizza la proposta di renderla **dinamica** durante le iterazioni (Byrd, Chin, Nocedal e Wu, 2012).

## Stato dell'arte: SGD e il problema della varianza

- SGD (Robbins–Monro, 1951): aggiornamento su un singolo esempio (o un piccolo batch),  $w_{k+1} = w_k - \alpha_k \nabla \ell(w_k; i_k)$ , con  $\mathbb{E}[\nabla \ell(w_k; i)] = \nabla J(w_k)$ : stimatore *non distorto*, costo per iterazione indipendente da  $N$ .
- Ma la varianza  $\text{Var}[\nabla \ell(w; i)]$  **non** tende a zero vicino all'ottimo: quando  $\nabla J(w) \rightarrow 0$  il rumore domina e limita la precisione raggiungibile.
- Conseguenze pratiche: a passo fisso si resta in un intorno dell'ottimo (precisione limitata); a passo  $\alpha_k \rightarrow 0$  la convergenza rallenta.
- Su larga scala conta il **costo totale** (costo per iterazione  $\times$  numero di iterazioni), non solo il numero di iterazioni.
- Due strade per uscirne: aumentare il campione (costo) oppure ridurre la varianza con stimatori migliori.

## Metodi a varianza ridotta: l'idea comune

- SVRG (*Stochastic Variance Reduced Gradient*; Johnson–Zhang, 2013) e SAGA (metodo di riduzione della varianza di Defazio–Bach–Lacoste-Julien, 2014): SGD + *correzione* costruita su quantità globali.
- Forma generale dello stimatore corretto:

$$g_k = \nabla \ell(w_k; i_k) - c_k + \mathbb{E}[c_k],$$

che resta non distorto:  $\mathbb{E}[g_k] = \nabla J(w_k)$ .

- Se la correzione  $c_k$  è correlata al rumore del gradiente stocastico, la varianza di  $g_k$  si riduce (idealmente tende a zero vicino all'ottimo): *variance reduction*.
- Differenza chiave tra i due metodi:
  - **SVRG**: ricalcola periodicamente un gradiente esatto su un punto di ancoraggio;
  - **SAGA**: mantiene una tavola dei gradienti per-esempio, aggiornata a ogni iterazione.

Dalla tesi — Sez. 4.1 (Lavori Correlati) e Appendice D (Schemi dei metodi a varianza ridotta)

# SVRG: l'ancora e il ricampionamento ogni $t$ iterazioni

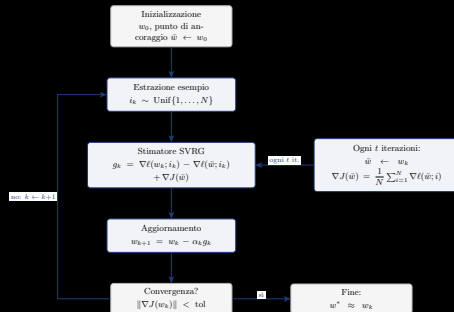
- Punto di ancoraggio  $\bar{w}$  con gradiente esatto  $\nabla J(\bar{w})$ , ricalcolato ogni  $t$  iterazioni.

- Stimatore non distorto:

$$g_k = \nabla \ell(w_k; i_k) - \nabla \ell(\bar{w}; i_k) + \nabla J(\bar{w})$$

- **Idea:** finché  $w_k \approx \bar{w}$ , per la regolarità dei gradienti  $\nabla \ell(w_k; i_k) \approx \nabla \ell(\bar{w}; i_k)$ : i due termini stocastici si elidono e resta  $g_k \approx \nabla J(\bar{w})$ : poca varianza.
- Quando  $w_k$  si allontana da  $\bar{w}$  la correzione “invecchia”: il rumore non è più eliso e la varianza ricresce  $\rightarrow$  si aggiorna l'ancora e si ricampiona l'intero dataset.

Dalla tesi — Appendice D, Schema SVRG



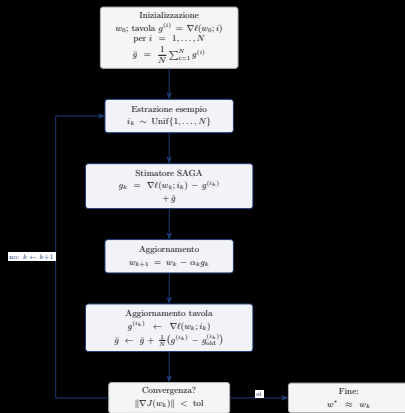
# SAGA: la tavola dei gradienti, nessun ricalcolo globale

- Tavola  $g^{(i)}$ : l'ultimo gradiente calcolato per ogni esempio  $i$ ; media  $\bar{g} = \frac{1}{N} \sum_i g^{(i)}$  (solo memoria  $O(Nm)$ , nessuna passata completa periodica).
- Stimatore:

$$g_k = \nabla \ell(w_k; i_k) - g^{(i_k)} + \bar{g}$$

- A ogni iterazione si aggiorna *solo* la voce estratta:  $g^{(i_k)} \leftarrow \nabla \ell(w_k; i_k)$  e  $\bar{g}$  di conseguenza.
- Stesso schema di riduzione della varianza di SVRG, ma la correzione è sempre “fresca” per l'esempio appena campionato.

Dalla tesi — Appendice D, Schema SAGA (fig. saga)



## Limiti di SVRG/SAGA e conseguenze per la tesi

- **SVRG**: memoria extra minima (solo l'ancora e il suo gradiente,  $O(m)$ ); ma ogni  $t$  iterazioni va ricalcolato il gradiente esatto  $\nabla J(\bar{w})$  con una passata completa sul dataset: costo *temporale*  $O(Nm)$  per passata, che per  $N$  molto grande può dominare il costo complessivo.
- **SAGA**: nessuna passata completa periodica, ma *memoria*  $O(Nm)$  per la tavola dei gradienti (un gradiente per ogni esempio): improponibile quando  $N \cdot m$  è molto grande.
- In entrambi la dimensione del campione è **fissata a priori** (un esempio per iterazione): non è mai adattata alle necessità di accuratezza del momento.
- Le garanzie di convergenza lineare valgono sotto convessità forte.
- **Conclusione**: rendere *dinamica* la dimensione del campione, aumentandola (e ricampionando) quando la stima corrente non è abbastanza accurata: è l'approccio di Byrd, Chin, Nocedal e Wu (2012), al centro della tesi.

## La varianza del gradiente stimato: definizioni

- Il gradiente sul batch è la *media* dei gradienti per-esempio,  
 $g_k = \nabla J_{S_k}(w_k) = \frac{1}{n_k} \sum_{i \in S_k} \nabla \ell(w_k; i)$ , con  $\mathbb{E}[g_k] = \nabla J(w_k)$  (stimatore non distorto).  
L'errore  $e_k = g_k - \nabla J(w_k)$  non è calcolabile; si controlla la *varianza dello stimatore*:  
 $\mathbb{E}[\|e_k\|_2^2] = \|\text{Var}(g_k)\|_1$  (somma delle varianze, componente per componente).
- Definizioni:  $\mathcal{V}$  è la *varianza della popolazione* del gradiente della loss (non calcolabile: servirebbe  $\nabla J(w_k)$ );  $\widehat{\mathcal{V}}$  (“v-hat”) è la sua stima *campionaria* sul batch corrente, centrata sulla media campionaria  $g_k$ :

$$\mathcal{V} := \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2, \quad \widehat{\mathcal{V}} := \frac{1}{n_k - 1} \sum_{i \in S_k} (\nabla \ell(w_k; i) - g_k)^2,$$

quadrati e divisioni sempre componente per componente ( $\in \mathbb{R}^m$ ).

Dalla tesi — Sez. 5.1



## Varianza dello stimatore: decomposizione

- $\widehat{\mathcal{V}}$  (denominatore  $n_k - 1$ ) è non distorta,  $\mathbb{E}[\widehat{\mathcal{V}}] = \mathcal{V}$ : stima la *dispersione dei singoli gradienti* del batch. Ma  $g_k$  è la loro *media*; la varianza della media si ricava da quella dei singoli con la *decomposizione* (con reinserimento, indipendenza):

$$\text{Var}(g_k) = \text{Var}\left(\frac{1}{n_k} \sum_{i \in S_k} \nabla \ell(w_k; i)\right) = \frac{1}{n_k^2} \sum_{i \in S_k} \text{Var}(\nabla \ell(w_k; i)) = \frac{\mathcal{V}}{n_k}.$$

- Senza reinserimento (i batch reali) si aggiunge il *fattore di correzione per popolazione finita*:

$$\text{Var}(g_k) = \frac{\mathcal{V}}{n_k} \cdot \frac{N - n_k}{N - 1}.$$

La sua dimostrazione è in Appendice (tesi). Per  $N \gg n_k$  (condizione tipica nel machine learning) il fattore è  $\approx 1$ , quindi vale la forma pratica  $\text{Var}(g_k) \approx \mathcal{V}/n_k$ .

Dalla tesi — Sez. 5.1; dimostrazione del fattore di correzione per popolazione finita in Appendice

## Campionamento dinamico: la Condizione di Controllo della Varianza

- Si parte da un batch piccolo: iterazioni economiche; il batch cresce solo quando la stima del gradiente non è abbastanza accurata.
- Criterio (CCV): la varianza stimata dello stimatore del gradiente deve essere piccola rispetto alla norma del gradiente stimato:

$$\frac{\|\widehat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2, \quad \theta \in (0, 1)$$

- Se la CCV è violata si ricampiona con un batch più grande:

$$n_k^{\text{new}} = \left\lceil \frac{\|\widehat{\mathcal{V}}\|_1}{\theta^2 \|g_k\|_2^2} \right\rceil$$

- Profilo a “scalini”;  $\theta$  piccolo  $\rightarrow$  si aumenta presto (batch accurati),  $\theta$  grande  $\rightarrow$  batch piccolo più a lungo.
- Nessuna stima di  $\kappa$  né regola geometrica a priori: la crescita è guidata dai dati.

Dalla tesi — Sez. 5.1 (Byrd, Chin, Nocedal e Wu, 2012)

## Direzione di discesa: livello puntuale e garanzia in media

- **Livello puntuale (deterministico).** Se per il batch corrente vale la condizione di accuratezza  $\|e_k\|_2 \leq \theta \|g_k\|_2$  ( $e_k = g_k - \nabla J(w_k)$ ), allora  $-g_k$  è di discesa per  $J$ : per Cauchy–Schwarz,

$$\nabla J(w_k)^\top g_k = \|g_k\|_2^2 - e_k^\top g_k \geq \|g_k\|_2^2 - \|e_k\|_2 \|g_k\|_2 \geq (1 - \theta) \|g_k\|_2^2 > 0.$$

È una condizione *sufficiente e puntuale*: ipotesi dell'analisi deterministica, non una conseguenza della CCV.

- **Livello CCV (in aspettativa).** La CCV controlla la *varianza* dello stimatore ( $\|\widehat{\mathcal{V}}\|_1/n_k \leq \theta^2 \|g_k\|_2^2$ ), non l'errore del singolo mini-batch: non garantisce la discesa a ogni iterazione.
- **Garanzia in media.**  $g_k$  è non distorto,  $\mathbb{E}[g_k] = \nabla J(w_k)$ , quindi

$$\mathbb{E}[\nabla J(w_k)^\top g_k] = \nabla J(w_k)^\top \mathbb{E}[g_k] = \|\nabla J(w_k)\|_2^2 > 0,$$

e la CCV tiene sotto controllo le fluttuazioni (base dell'analisi di convergenza in media).

Dalla tesi — Sez. 5.1: condizione puntuale (deterministica), CCV (in aspettativa) e garanzia in media

# I quattro algoritmi a campionamento dinamico

- **Dynamic GD**: gradiente su campione dinamico con CCV e line search di Wolfe (base del metodo).
- **Newton-CG**: Hessiana sottocampionata ( $|\mathcal{H}| = R |\mathcal{S}|$ ) e criterio di arresto adattivo del CG interno, basato sulla varianza dei prodotti Hessiana-vettore: niente iterazioni interne sprecate quando l'Hessiana è limitata dal sottocampionamento.
- **Newton-CG  $L_1$** : gradiente generalizzato, identificazione dell'active set e line search proiettata (soluzioni sparse).
- **BB-CCV**: passo di Barzilai–Borwein con campionamento dinamico (line search di Armijo).
- Tutti implementati in Python (Appendice B) e nell'app web interattiva; per ognuno, lo schema seguente mostra il flusso con la CCV come decisione centrale.

## Il passo $\alpha_k$ : condizioni di Armijo e Wolfe

- Il passo  $\alpha_k$  è scelto da una *line search* lungo la direzione di discesa  $p$  ( $\nabla f(x_k)^T p < 0$ ):  $\alpha$  deve ridurre  $f$  in modo “sufficiente”, senza essere troppo piccolo.
- **Armijo**: limite *superiore* su  $f$  lungo  $p$ : il passo è accettato se  $f(x_k + \alpha p)$  sta sotto la retta  $f(x_k) + c_1 \alpha \nabla f(x_k)^T p$ :

$$f(x_k + \alpha p) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p, \quad c_1 \in (0, 1) \text{ (tip. } 10^{-4}\text{)},$$

la retta ha pendenza  $c_1 \nabla f(x_k)^T p$ , meno ripida della tangente in  $\alpha = 0$ : i passi che la superano vengono rifiutati e ridotti (impedisce i passi *troppo grandi*).

- **Wolfe**: condizione di *curvatura* (limite *inferiore* sulla derivata lungo la direzione). Con  $g(\alpha) = f(x_k + \alpha p)$  e  $g'(\alpha) = \nabla f(x_k + \alpha p)^T p$ :

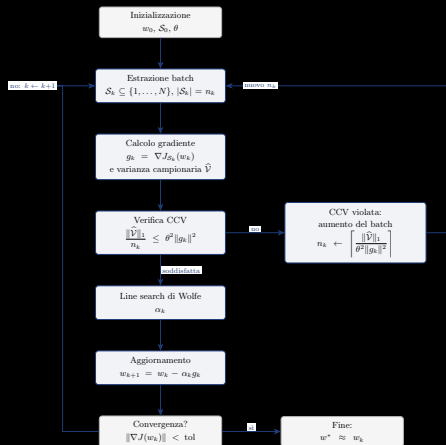
$$g'(\alpha) \geq c_2 g'(0), \quad c_2 \in (c_1, 1), \quad g'(0) = \nabla f(x_k)^T p < 0,$$

poiché  $c_2 < 1$ ,  $c_2 g'(0)$  è meno negativo di  $g'(0)$ : la condizione scarta i passi *troppo piccoli* (per  $\alpha \rightarrow 0^+$ ,  $g'(\alpha) \rightarrow g'(0)$ , che viola la disuguaglianza).

- $0 < c_1 < c_2 < 1$  rende le due condizioni *compatibili* (intervallo non vuoto di  $\alpha$  accettabili); se  $c_1 \geq c_2$  le condizioni possono diventare contraddittorie.

Dalla tesi — Appendice A; nei codici (Appendice B): Wolfe con  $c_1=10^{-4}$ ,  $c_2=0.9$  per Dynamic GD e Newton-CG; Armijo con backtracking per BB-CCV; Armijo proiettata per Newton-CG  $L_1$

# Schema: Dynamic GD (Fig. 5.1)



Flusso CCV: se la varianza campionaria è troppo grande rispetto a  $\theta^2 \|g_k\|^2$ ,  $n_k$  cresce e si ricampiona; altrimenti line search di Wolfe e aggiornamento.

## Newton-CG: struttura del metodo (eq. 5.31–5.32)

- Due campioni a ogni iterazione:  $\mathcal{S}_k$  per il **gradiente** (campionamento dinamico con CCV) e  $\mathcal{H}_k \subset \mathcal{S}_k$  per l'**Hessiana**, con  $|\mathcal{H}_k| \ll |\mathcal{S}_k|$  (prodotti Hessiana-vettore economici).
- La direzione di ricerca  $d_k$  è la soluzione approssimata del sistema (Hessian-free: la matrice non è mai costruita)

$$\nabla^2 J_{\mathcal{H}_k}(w_k) d = -\nabla J_{\mathcal{S}_k}(w_k).$$

- Rapporto di sottocampionamento fissato a priori e costante; quando  $\mathcal{S}_k$  cresce (CCV) cresce in proporzione anche  $\mathcal{H}_k$ :

$$R = \frac{|\mathcal{H}_k|}{|\mathcal{S}_k|} < 1, \quad |\mathcal{H}_k| = \lceil R |\mathcal{S}_k| \rceil.$$

Linea guida: il costo dei prodotti Hessiana-vettore nel CG non deve superare quello di  $\nabla J_{\mathcal{S}_k}(w_k)$ .

Dalla tesi — Sez. 5.2, eq. 5.31–5.32

## Newton-CG: non serve un residuo più piccolo dell'errore di Hessiana

- Il CG risolve in modo **approssimato** il sistema di Newton (eq. 5.31), gradiente su  $\mathcal{S}_k$  e Hessiana su  $\mathcal{H}_k$ ; il suo **residuo** è (eq. 5.33):

$$r_k = \nabla^2 J_{\mathcal{H}_k}(w_k) d + \nabla J_{\mathcal{S}_k}(w_k),$$

- L'Hessiana è quindi **approssimata**: il residuo del sistema che userebbe il campione grande  $\mathcal{S}_k$  per gradiente ed Hessiana si scompone in

$$\nabla^2 J_{\mathcal{S}_k}(w_k) d + \nabla J_{\mathcal{S}_k}(w_k) = \underbrace{r_k}_{\text{residuo del CG}} + \underbrace{\Delta_{\mathcal{H}_k}(w_k; d)}_{\text{errore di Hessiana}},$$

dove  $\Delta_{\mathcal{H}_k}(w_k; d) = [\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)]d$  è **costante durante il CG** e **non eliminabile** (il campione  $\mathcal{H}_k$  è fissato).

- Poiché  $\Delta_{\mathcal{H}_k}$  non si può eliminare, *non serve* ridurre  $r_k$  fino a zero: **basta fermarlo quando è confrontabile con  $\Delta_{\mathcal{H}_k}$** ; oltre, le iterazioni del CG sono *tempo sprecato*.
- Perché non è calcolabile?**  $\Delta_{\mathcal{H}_k}$  contiene il prodotto Hessiana–vettore  $\nabla^2 J_{\mathcal{S}_k}(w_k)d$ , la media su *tutto*  $\mathcal{S}_k$  dei prodotti  $\nabla^2 \ell(w_k; i)d$ : calcolarlo costerebbe quanto l'Hessiana piena e **vanificherebbe** il sottocampionamento di  $\mathcal{H}_k$ . Come per l'errore  $e_k$  del gradiente: stima con la **varianza campionaria** dei prodotti Hessiana–vettore su  $\mathcal{H}_k$  (prossima slide).

Dalla tesi — Sez. 5.2, eq. 5.31 e 5.33–5.34 e discussione del criterio



## Newton-CG: come si stima l'errore di Hessiana (eq. 5.35)

- $\Delta_{\mathcal{H}_k}(w_k; d)$  **non è calcolabile** (servirebbe il prodotto Hessiana–vettore su tutto  $\mathcal{S}_k$ ): come per l'errore del gradiente, se ne stima il valore atteso con la **varianza campionaria dei prodotti Hessiana–vettore** su  $\mathcal{H}_k$  (campionamento senza reinserimento: fattore  $(|\mathcal{S}_k| - |\mathcal{H}_k|)/(|\mathcal{S}_k| - 1) \approx 1$  perché  $|\mathcal{H}_k| \ll |\mathcal{S}_k|$ ):

$$\mathbb{E}[\|\Delta_{\mathcal{H}_k}(w_k; d)\|_2^2] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1}{|\mathcal{H}_k|}.$$

Come per il gradiente: è la varianza della *media* su  $\mathcal{H}_k$  (la dispersione dei singoli prodotti divisa per  $|\mathcal{H}_k|$ ).

- La stima vale per una direzione generica  $d$ : nel CG andrebbe rivalutata a ogni iterazione interna  $j$ , sulla direzione corrente  $d_j$ . Ricalcolare la varianza campionaria a ogni passo è però costoso: la prossima slide mostra come si evita.

Dalla tesi — Sez. 5.2, eq. 5.35

## Newton-CG: omogeneità di grado 2 e un solo calcolo (eq. 5.36–5.38)

- Su un campione fissato, la varianza dei prodotti Hessiana–vettore è una **forma quadratica** in  $d$ :  
 $\text{Var}(\nabla^2 \ell(w_k; i) d) = d^T \Sigma_{\mathcal{H}_k} d$  ( $\Sigma_{\mathcal{H}_k}$  covarianza campionaria sdp; la relazione vale componente per componente). Vale quindi l'**omogeneità di grado 2**

$$\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i)(\lambda d)) = \lambda^2 \text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d).$$

- **Un solo calcolo.** Per non ricalcolare la varianza a ogni iterazione del CG (costo elevato), la si calcola **una sola volta** sulla prima direzione  $p_0 = -g_k$ ,

$$\alpha := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2} = \frac{\|p_0^T \Sigma_{\mathcal{H}_k} p_0\|_1}{\|p_0\|_2^2},$$

e per ogni  $d_j$  si usa  $\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d_j)\|_1 \approx \alpha \|d_j\|_2^2$ .

- **Approssimazione.** La varianza effettiva di  $d_j$  è  $\|d_j^T \Sigma_{\mathcal{H}_k} d_j\|_1$ , non  $\alpha \|d_j\|^2$ :  $\alpha$  è la media pesata degli autovalori di  $\Sigma_{\mathcal{H}_k}$  ( $\lambda_{\min} \leq \alpha \leq \lambda_{\max}$ ) e usarla su tutte le direzioni è lecito se la forma quadratica è sufficientemente *ben condizionata* (dipendenza angolare trascurabile).
- Dalla (5.35), la soglia per il test di arresto del CG è  $\gamma := \alpha/|\mathcal{H}_k|$  e  $\Psi(d) := \gamma \|d\|_2^2$ : per ogni  $d_j$ ,  
 $\Psi(d_j) \approx \mathbb{E} \|\Delta_{\mathcal{H}_k}(w_k; d_j)\|_2^2$  **cresce con**  $\|d_j\|^2$  (prossima slide: criterio di arresto).

Dalla tesi — Sez. 5.2, eq. 5.36–5.38

## Newton-CG: criterio di arresto adattivo del CG (eq. 5.39)

- A ogni iterazione interna  $j$  del CG si confronta il residuo  $r_{j+1}$  con la stima  $\Psi(d_j)$  dell'errore di Hessiana:

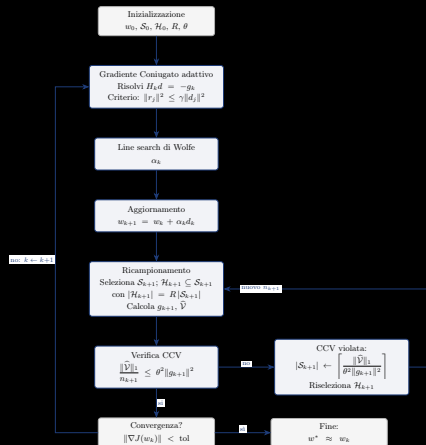
$$\|r_{j+1}\|_2^2 \leq \Psi(d_j) = \gamma \|d_j\|_2^2.$$

Se la condizione è verificata il CG **termina** e si accetta la direzione  $d_{j+1}$ ; altrimenti si prosegue con l'iterazione successiva.

- È un criterio *automatico*:  $\gamma$  è fissata dai dati all'inizio del CG e la soglia  $\Psi(d_j)$  si aggiorna a ogni iterazione con la sola  $\|d_j\|_2^2$  — nessuna tolleranza da scegliere a priori.
- **Perché è adattivo.**
  - evita calcoli inutili quando l'approssimazione dell'Hessiana è limitata dal campione  $\mathcal{H}_k$ ;
  - $\Psi \propto \|r_0\|^2$ : la soglia segue la *distanza dal minimo*;
  - $\Psi$  cresce con  $\|d_j\|^2$ : ferma il CG prima che la direzione diventi eccessivamente grande (stabilità numerica).

Dalla tesi — Sez. 5.2, eq. 5.39 e discussione del criterio

## Schema: Newton-CG (Fig. 5.2)



Hessiana sottocampionata con  $|\mathcal{H}| = R |\mathcal{S}|$ ; il CG interno è Hessian-free e si ferma con un criterio adattivo basato sulla varianza dei prodotti Hessiana-vettore.

## Newton-CG $L_1$ : $z_k$ e la ricerca lineare proiettata

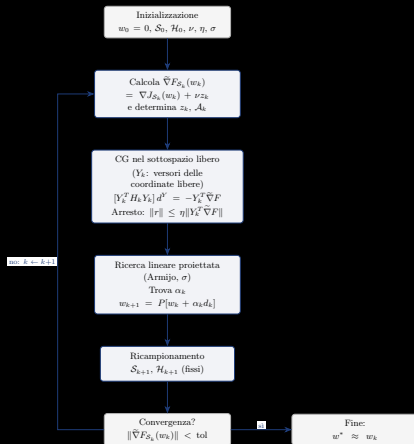
- $F = J + \nu \|w\|_1$  **non è differenziabile** dove  $w_i = 0$ : l'algoritmo deve decidere se *liberare*  $w_i$  o *attivarla* (tenerla a zero, soluzione sparsa).
- Il vettore  $z_k \in \{-1, 0, +1\}^m$  (eq. 5.44) **codifica la decisione** su ogni variabile:  $z_k^i = \text{sign}(w_k^i)$  se  $w_k^i \neq 0$ ; se  $w_k^i = 0$ , vale  $\pm 1$  quando  $|\partial J / \partial w_i| > \nu$  (conviene liberare) e 0 altrimenti (resta nell'**active set**  $\mathcal{A}_k = \{i : z_k^i = 0\}$ , eq. 5.46).
- $z_k$  individua la **faccia ortante**  $\Omega_k = \{d : \text{sign}(d^i) = \text{sign}(z_k^i)\}$  (eq. 5.45), dove  $\|w\|_1$  è lineare e il gradiente generalizzato è  $\tilde{\nabla} F = \nabla J + \nu z_k$ ; su  $\Omega_k$  si minimizza il modello quadratico ristretto alle variabili libere.
- **Perché serve la ricerca lineare proiettata.** Il candidato  $w_k + \alpha d_k$  può uscire da  $\Omega_k$ : una variabile dell'active set lascerebbe lo zero o una libera attraverserebbe l'angolo della  $L_1$ . La proiezione  $P(\cdot)$  (eq. 5.49) azzera le componenti con segno non coerente con  $z_k$ ; il passo è scelto con Armijo su  $F$  (eq. 5.50) e

$$w_{k+1} = P[w_k + \alpha_k d_k],$$

così le variabili attive restano a zero e la soluzione è sparsa.

Dalla tesi — Sez. 5.3, eq. 5.44–5.50

## Schema: Newton-CG $L_1$ (Fig. 5.5)



Gradiente generalizzato per la  $L_1$ : identificazione dell'active set e line search proiettata mantengono a zero le variabili giuste, producendo soluzioni sparse.

## Il passo di Barzilai–Borwein (1/2) — relazione secante (eq. 5.52–5.54)

- BB-CCV sostituisce il passo *fisso* del Dynamic GD con un passo **adattivo** che stima la curvatura di  $J$  lungo la direzione percorsa usando **solo gradienti già calcolati**: nessuna Hessiana.
- Tra due iterate consecutive,  $w_{k+1} = w_k + \Delta w_k$ , lo sviluppo al primo ordine del gradiente  $g = \nabla J$  attorno a  $w_k$  è

$$g(w_{k+1}) \approx g(w_k) + \nabla^2 J(w_k) \Delta w_k.$$

- Posto  $s_k := w_{k+1} - w_k$  (spostamento) e  $y_k := g(w_{k+1}) - g(w_k)$  (variazione del gradiente), si ha la **relazione secante**

$$y_k \approx \nabla^2 J(w_k) s_k.$$

- I metodi quasi-Newton approssimano l'Hessiana con  $B_{k+1}$  imponendo  $B_{k+1} s_k = y_k$ : è il punto di partenza del passo BB (prossima slide).

Dalla tesi — Sez. 5.4, eq. 5.52–5.54

## Il passo di Barzilai–Borwein (2/2) — minimi quadrati e salvaguardia (eq. 5.55–5.56)

- BB spinge il quasi-Newton all'estremo:  $B_{k+1} = \alpha^{-1} I$  (passo **scalare**); la secante diventa  $\alpha^{-1} s \approx y$ , un sistema di  $n$  equazioni in un'incognita risolto **ai minimi quadrati**. Le due formule classiche:

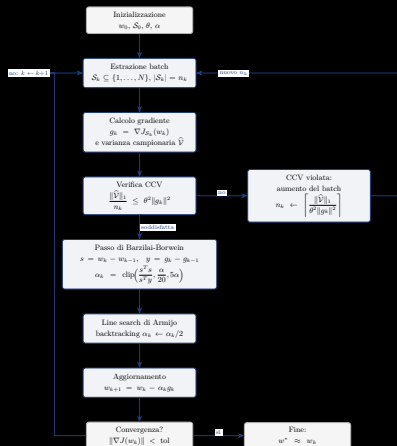
$$\alpha_k^{(1)} = \frac{s_k^\top y_k}{y_k^\top y_k} \quad (\min \|\alpha y - s\|^2), \quad \alpha_k^{(2)} = \frac{s_k^\top s_k}{s_k^\top y_k} \quad (\min \|\alpha^{-1} s - y\|^2).$$

- BB-CCV usa la **seconda**, più *aggressiva*: passi più grandi dove il gradiente cambia poco (direzione piatta), più piccoli dove cambia molto.
- Rischio**: se  $s^\top y \approx 0$  (spostamento e variazione del gradiente quasi perpendicolari) il passo esplode; la **salvaguardia**  $\text{clip}(s^\top s / s^\top y, \alpha/20, 5\alpha)$  lo tiene in  $[\alpha/20, 5\alpha]$ .
- Su  $J$  quadratiche ( $y = As$  esatta,  $A$  definita positiva) il passo è **ottimale**: poche iterazioni. Su  $J$  generali la secante è solo locale e  $s^\top y$  può diventare  $\leq 0$ : stabilizzano salvaguardia e line search di Armijo.

Dalla tesi — Sez. 5.4, eq. 5.55–5.56 e salvaguardia clip



## Schema: BB-CCV (Fig. 5.6)



Gradiente dinamico con passo Barzilai–Borwein (line search di Armijo): la CCV regola la dimensione del campione mentre il passo adatta la lunghezza dello spostamento.

## App web interattiva: esperimenti nel browser

- `visualizzazione.html`: singolo file, Pyodide + Plotly, nessuna installazione o server.
- Codice Python dei quattro algoritmi visibile e modificabile nel browser; preset quadratici + dataset sintetico “centrato” ( $\widehat{J}_N = J$ ).
- Percorso 3D/2D su  $J(w)$ , metriche in tempo reale, pannelli “Analisi” e “Test batch” (per riprodurre le tabelle della tesi).
- Permette di osservare la dinamica del batch ( $n_k$  vs  $k$ ) e l’effetto del parametro  $\theta$  sulla soglia CCV.
- Implementazione Python standalone equivalente (`simulazione_batch.py`) per la rigenerazione di figure e tabelle della tesi.

# Le funzioni test: espressioni usate negli esperimenti

- **Ben condizionata** ( $\kappa \approx 1.1$ ):  $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.1 w_1 w_2$
- **Mal condizionata** ( $\kappa \approx 20$ ):  $J(w) = 20 (w_1 - 1)^2 + (w_2 + 2)^2$
- **Molto mal condizionata** ( $\kappa \approx 100$ ):  $J(w) = 100 (w_1 - 1)^2 + (w_2 + 2)^2$
- **Termine incrociato** ( $\kappa \approx 1.67$ ):  $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.5 (w_1 - 1)(w_2 + 2)$
- **Funzione di Rosenbrock** (non quadratica,  $c=100$ ):

$$J(w) = \frac{1}{N} \sum_{i=1}^N \left[ (w_1 - a_i)^2 + 100((w_2 - b_i) - (w_1 - a_i))^2 \right],$$

con  $a_i, b_i$  stocastici centrati ( $\sigma = 0.2$ , medie 1 e -2),  $N = 200$ , seed 42:  $w_* \approx (1.06, -1.96)$ .

- Minimo comune dei quattro problemi quadratici:  $w_* = (1, -2)$  (dataset centrato,  $\hat{J}_N = J$ ).

Dalla tesi — Sez. 6.1 (Setup Sperimentale)

## Setup e risultati numerici

- Setup:  $N = 200$ , seed 42,  $w_0 = (2, -3)$ ,  $\alpha = 0.1$ ,  $\theta = 0.5$ ,  $n_0 = 5$ , 30 iterazioni, tolleranza  $\|\nabla J\| < 10^{-6}$ .
- Preset: ben condizionato ( $\kappa \approx 1.1$ ), mal condizionato ( $\kappa \approx 20$ ), molto mal condizionato ( $\kappa \approx 100$ ), incrociato.
- Profilo a “scalini” di  $n_k$  confermato dagli esperimenti: il batch cresce solo quando la CCV lo richiede.
- BB-CCV raggiunge la precisione macchina ( $\approx 1.1 \times 10^{-14}$ ) su  $\kappa \approx 100$ ; l’analisi teorica ( $\kappa$  al posto di  $\kappa^2$ ) è confermata dai dati.

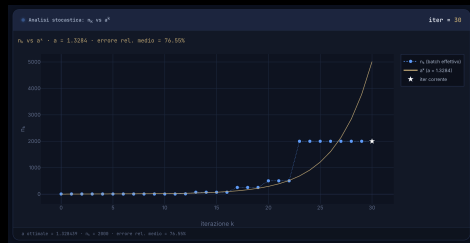
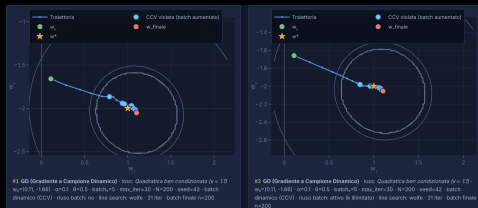


Fig. 5.3 — dimensione del campione  $n_k$  vs iterazioni  $k$

## Riuso del mini-batch (Sez. 6.5)

- Variante studiata nella tesi: riusare lo stesso campione per  $M$  iterazioni consecutive (ricalcolando però il gradiente al punto corrente, non riusando i gradienti vecchi).
- La CCV fa da segnale di “esaurimento” del campione: quando non è più soddisfatta,  $n_k$  cresce e si ricampiona.
- $M = \infty$ : si tiene il campione finché la CCV vale; il costo per iterazione scende, ma il campione “invecchia”.
- Effetti dipendenti dal metodo e dal condizionamento del problema: quantificati con le tabelle del Cap. 6.



Traiettorie con riuso del batch

# Cosa significa riusare il batch

- Riusare il batch = tenere **fissa solo la lista di indici**  $S_k$  per  $M$  iterazioni; il gradiente è **ricalcolato al punto corrente**  $w_k$  a ogni iterazione (non si riusa il gradiente né una “direzione vecchia”).
- Sui *ben condizionati* la traiettoria sembra “dritta” perché  $w_S^*$  è vicino a  $w^*$  in ogni direzione: i gradienti successivi puntano quasi allo stesso bersaglio.
- Sui *mal condizionati* il minimo campionario  $w_S^*$  è spostato lungo la direzione quasi piatta: il batch fisso “tira” verso un bersaglio sbagliato (**bias sistematico**, Sez. 6.5.2), che la CCV **non** vede (misura la varianza interna, non il bias).
- **Primo ordine** (Dynamic GD, BB-CCV):  $d_k = -g_k$  è sempre opposto al gradiente fresco (angolo  $180^\circ$ ), la discesa sul batch è garantita: il rischio è solo la lenta *deriva* verso  $w_S^*$ .
- **Secondo ordine** (Newton-CG): con  $\mathcal{H}_k$  fisso la curvatura “invecchia” mentre  $w_k$  avanza;  $d_k = -H_k^{-1}g_k$  può finire a  $\approx 90^\circ$  dalla vera discesa e la line search taglia il passo  $\Rightarrow$  per questo serve lo stop adattivo (slide seguenti).

Dalla tesi — Sez. 6.5.2, “Meccanismo: vantaggi e limiti del riuso”

# Riuso del mini-batch: sintesi dei risultati (Tab. 6.1)

- **Aiuta:** ben condizionato ( $\kappa \approx 1.1$ ), termine incrociato, BB-CCV su  $\kappa \approx 20$  (fino a  $\sim 10^{-10}$ ).
- **Peggiora:** BB-CCV su  $\kappa \approx 100$  (base già alla precisione macchina,  $\sim 10^{-14}$ ) e Dynamic GD/Newton-CG sui mal condizionati con  $M=\infty$  (direzioni che invecchiano).

| Metodo                                                 | base                     | $M=\infty$                                 | $M=10$                                     | $M=5$                                      | $M=3$                                      | $M=2$                                      | $M=1$                                      |
|--------------------------------------------------------|--------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|
| $\kappa \approx 1.1$ - Dynamic GD                      | $1.1895 \times 10^{-1}$  | $1.1739 \times 10^{-1} \blacktriangle$     | $1.1739 \times 10^{-1} \blacktriangle$     | $1.1920 \times 10^{-1} \blacktriangledown$ | $1.1903 \times 10^{-1} \blacktriangledown$ | $1.1777 \times 10^{-1} \blacktriangle$     | $1.1895 \times 10^{-1} =$                  |
| $\kappa \approx 1.1$ - BB-CCV                          | $1.1662 \times 10^{-1}$  | $1.1662 \times 10^{-1}$                    | $1.1662 \times 10^{-1}$                    | $1.1662 \times 10^{-1}$                    | $1.1662 \times 10^{-1} \blacktriangle$     | $1.1662 \times 10^{-1} \blacktriangle$     | $1.1662 \times 10^{-1} =$                  |
| $\kappa \approx 1.1$ - Newton-CG                       | $4.0264 \times 10^{-1}$  | $2.2230 \times 10^{-1}$                    | $3.7373 \times 10^{-1}$                    | $4.1553 \times 10^{-1} \blacktriangledown$ | $3.2913 \times 10^{-1}$                    | $2.6069 \times 10^{-1}$                    | $3.0560 \times 10^{-1} \blacktriangle$     |
| $\kappa \approx 1.1$ - Newton-CG $L_1$                 | $1.0167 \times 10^{-1}$  | $8.1750 \times 10^{-2}$                    | $9.2407 \times 10^{-2}$                    | $1.0010 \times 10^{-1}$                    | $1.0624 \times 10^{-1} \blacktriangledown$ | $9.4078 \times 10^{-2}$                    | $1.0028 \times 10^{-1} \blacktriangle$     |
| $\kappa \approx 20$ - Dynamic GD                       | $4.9050 \times 10^{-2}$  | $1.0103 \times 10^{-1} \blacktriangledown$ | $7.6020 \times 10^{-2} \blacktriangledown$ | $4.7211 \times 10^{-2}$                    | $1.1746 \times 10^{-1} \blacktriangledown$ | $4.2898 \times 10^{-2} \blacktriangle$     | $4.9050 \times 10^{-2} =$                  |
| $\kappa \approx 20$ - BB-CCV                           | $9.7195 \times 10^{-5}$  | $9.5364 \times 10^{-10}$                   | $9.5364 \times 10^{-10}$                   | $2.6835 \times 10^{-3}$                    | $3.8422 \times 10^{-8}$                    | $1.5436 \times 10^{-10}$                   | $9.7195 \times 10^{-5} =$                  |
| $\kappa \approx 20$ - Newton-CG                        | $2.3463 \times 10^{-1}$  | $1.2807 \times 10^0$                       | $9.4933 \times 10^{-1}$                    | $4.1051 \times 10^{-1}$                    | $3.3396 \times 10^{-1}$                    | $4.0617 \times 10^{-1} \blacktriangledown$ | $1.6683 \times 10^{-1} \blacktriangle$     |
| $\kappa \approx 20$ - Newton-CG $L_1$                  | $4.8802 \times 10^{-1}$  | $7.6221 \times 10^{-1}$                    | $5.5687 \times 10^{-1}$                    | $6.2190 \times 10^{-1}$                    | $4.5512 \times 10^{-1}$                    | $4.3392 \times 10^{-1} \blacktriangle$     | $5.4573 \times 10^{-1} \blacktriangledown$ |
| $\kappa \approx 100$ - Dynamic GD                      | $4.6668 \times 10^{-1}$  | $3.7060 \times 10^{-1}$                    | $3.7060 \times 10^{-1}$                    | $3.7060 \times 10^{-1}$                    | $3.7060 \times 10^{-1}$                    | $3.7060 \times 10^{-1}$                    | $4.6668 \times 10^{-1} =$                  |
| $\kappa \approx 100$ - BB-CCV                          | $1.0991 \times 10^{-14}$ | $7.4538 \times 10^{-3}$                    | $7.4538 \times 10^{-3}$                    | $7.4538 \times 10^{-3}$                    | $7.4538 \times 10^{-3}$                    | $7.4538 \times 10^{-3}$                    | $1.0991 \times 10^{-14} =$                 |
| $\kappa \approx 100$ - Newton-CG                       | $2.0969 \times 10^{-1}$  | $1.2807 \times 10^0$                       | $6.4441 \times 10^{-1}$                    | $3.3452 \times 10^{-1}$                    | $3.7338 \times 10^{-1}$                    | $4.4283 \times 10^{-1} \blacktriangledown$ | $2.3041 \times 10^{-1} \blacktriangledown$ |
| $\kappa \approx 100$ - Newton-CG $L_1$                 | $9.6954 \times 10^{-1}$  | $9.7065 \times 10^{-1}$                    | $8.6360 \times 10^{-1}$                    | $9.6997 \times 10^{-1}$                    | $8.7711 \times 10^{-1}$                    | $9.6944 \times 10^{-1}$                    | $8.6562 \times 10^{-1} \blacktriangle$     |
| incrociato ( $\kappa \approx 1.67$ ) - Dynamic GD      | $9.4161 \times 10^{-3}$  | $5.5315 \times 10^{-3}$                    | $9.9566 \times 10^{-3}$                    | $5.9490 \times 10^{-3}$                    | $7.5801 \times 10^{-3}$                    | $8.7213 \times 10^{-3}$                    | $9.4161 \times 10^{-3} =$                  |
| incrociato ( $\kappa \approx 1.67$ ) - BB-CCV          | $6.1644 \times 10^{-4}$  | $6.1689 \times 10^{-4}$                    | $6.1689 \times 10^{-4}$                    | $6.1689 \times 10^{-4}$                    | $6.1610 \times 10^{-4}$                    | $6.1685 \times 10^{-4}$                    | $6.1644 \times 10^{-4} =$                  |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG       | $3.5167 \times 10^{-1}$  | $4.1026 \times 10^{-2}$                    | $1.0511 \times 10^{-1}$                    | $3.3540 \times 10^{-1}$                    | $2.3364 \times 10^{-1}$                    | $2.1820 \times 10^{-1}$                    | $2.6381 \times 10^{-1} \blacktriangle$     |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG $L_1$ | $2.0877 \times 10^{-2}$  | $3.8938 \times 10^{-2}$                    | $3.9410 \times 10^{-2}$                    | $2.9304 \times 10^{-2}$                    | $2.3645 \times 10^{-2}$                    | $3.8543 \times 10^{-2} \blacktriangledown$ | $2.5154 \times 10^{-2} \blacktriangledown$ |
| Funzione di Rosenbrock ( $c=100$ ) - Dynamic GD        | $6.1354 \times 10^{-4}$  | $2.5435 \times 10^0 \blacktriangledown$    | $2.5435 \times 10^0 \blacktriangledown$    | $2.5435 \times 10^0 \blacktriangledown$    | $2.0803 \times 10^0 \blacktriangledown$    | $1.4213 \times 10^0$                       | $6.1354 \times 10^{-4} =$                  |
| Funzione di Rosenbrock ( $c=100$ ) - BB-CCV            | $1.0645 \times 10^{-1}$  | $1.6648 \times 10^{-1}$                    | $1.6648 \times 10^{-1}$                    | $1.6648 \times 10^{-1}$                    | $1.6648 \times 10^{-1}$                    | $3.6984 \times 10^{-1} \blacktriangledown$ | $1.0645 \times 10^{-1} =$                  |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG         | $2.0625 \times 10^{-1}$  | $1.0605 \times 10^0$                       | $1.1367 \times 10^0$                       | $1.2475 \times 10^0$                       | $1.1893 \times 10^0$                       | $1.2292 \times 10^0$                       | $9.3953 \times 10^{-1} \blacktriangledown$ |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG $L_1$   | $1.2525 \times 10^{-1}$  | $3.8199 \times 10^{-1}$                    | $2.4883 \times 10^{-1}$                    | $3.0766 \times 10^{-1}$                    | $3.1242 \times 10^{-1}$                    | $2.7111 \times 10^{-1}$                    | $2.0207 \times 10^{-1} \blacktriangledown$ |

Dalla tesi — Sez. 6.5.5 e Tab. 6.1: errore finale  $e_{30}$  (seed 42) per tutti i valori di  $M$ ;  $\blacktriangle/\blacktriangledown/=$  rispetto alla *base*.

## Robustezza del riuso su 5 seed indipendenti (Tab. 6.2)

- Su  $\kappa \approx 1.1$  il riuso regge su quasi tutti i seed (Dynamic GD e BB-CCV 5/5).
- Sui mal condizionati il segno dipende dal metodo; Newton-CG non migliora mai (0/5)  $\Rightarrow$  serve un criterio automatico di stop.

| Problema                             | Algoritmo       | $M=\infty$ |       |        | $M=10$ |       |        |
|--------------------------------------|-----------------|------------|-------|--------|--------|-------|--------|
|                                      |                 | Migl.      | Pegg. | Uguale | Migl.  | Pegg. | Uguale |
| $\kappa \approx 1.1$                 | Dynamic GD      | 5          | 0     | 0      | 5      | 0     | 0      |
|                                      | BB-CCV          | 5          | 0     | 0      | 5      | 0     | 0      |
|                                      | Newton-CG       | 4          | 1     | 0      | 3      | 2     | 0      |
|                                      | Newton-CG $L_1$ | 4          | 1     | 0      | 5      | 0     | 0      |
| $\kappa \approx 20$                  | Dynamic GD      | 2          | 3     | 0      | 1      | 4     | 0      |
|                                      | BB-CCV          | 3          | 2     | 0      | 3      | 2     | 0      |
|                                      | Newton-CG       | 0          | 5     | 0      | 0      | 5     | 0      |
|                                      | Newton-CG $L_1$ | 1          | 4     | 0      | 2      | 3     | 0      |
| $\kappa \approx 100$                 | Dynamic GD      | 3          | 2     | 0      | 3      | 2     | 0      |
|                                      | BB-CCV          | 1          | 3     | 1      | 1      | 3     | 1      |
|                                      | Newton-CG       | 0          | 5     | 0      | 0      | 5     | 0      |
|                                      | Newton-CG $L_1$ | 3          | 2     | 0      | 4      | 1     | 0      |
| incrociato ( $\kappa \approx 1.67$ ) | Dynamic GD      | 4          | 1     | 0      | 3      | 2     | 0      |
|                                      | BB-CCV          | 2          | 3     | 0      | 2      | 3     | 0      |
|                                      | Newton-CG       | 4          | 1     | 0      | 3      | 2     | 0      |
|                                      | Newton-CG $L_1$ | 2          | 3     | 0      | 1      | 4     | 0      |
| Funzione di Rosenbrock ( $c=100$ )   | Dynamic GD      | 2          | 3     | 0      | 2      | 3     | 0      |
|                                      | BB-CCV          | 1          | 4     | 0      | 1      | 4     | 0      |
|                                      | Newton-CG       | 1          | 4     | 0      | 2      | 3     | 0      |
|                                      | Newton-CG $L_1$ | 1          | 4     | 0      | 2      | 3     | 0      |

Dalla tesi — Sez. 6.5.5 e Tab. 6.2: numero di seed (su 5, {42, 7, 123, 2024, 999}) in cui  $e_{30}$  è minore/maggiore/uguale rispetto alla *base*.



## Stop adattivo con validation set: come funziona

- Il riuso richiede di fissare  $M$  (iterazioni sullo stesso batch), ma il valore ottimo cambia con problema e algoritmo e  $M=\infty$  può far collassare i metodi di Newton. Lo stop adattivo **decide da solo quando ricampionare**, senza scegliere  $M$  a priori.
- Si riserva una frazione  $p$  del dataset come *validation set*: il mini-batch è campionato dal solo training set (CCV con tetto  $|\mathcal{T}|$  invece di  $N$ ) e l'Hessiana segue il batch (modalità legata).

**Pseudocodice** (iterazione esterna  $k$ , batch in riuso):

1.  $S_k \leftarrow$  mini-batch dal training set;  $g_k = \nabla J_{S_k}(w_k)$ ; aggiorna  $w_{k+1}$  con la line search.
2. Ogni  $f$  iterazioni (dopo l'aggiornamento) valuta  $J_{\text{val}}(w_{k+1})$ , la loss media sul validation set.
3. *Progresso* se  $J_{\text{val}} \leq J_{\text{val}}^{\text{best}}(1 - \tau) - \epsilon_{\text{abs}}$ : aggiorna il best e azzera il contatore.
4. Se per  $P$  valutazioni consecutive *non* c'è progresso  $\Rightarrow$  **ricampiona** all'iterazione successiva (risplit se la strategia è *dinamica*).

Dalla tesi — Sez. 6.5.6. Iperparametri:  $P, f, p, \tau, \epsilon_{\text{abs}}$ , strategia di split (fissa o dinamica).

# Stop adattivo con validation set (Tab. 6.3)

- Ricampiona quando la loss su un *validation set* smette di migliorare per  $P$  valutazioni; evita il collasso di  $M=\infty$  sui metodi di Newton.
- Calibrato con ricerca a griglia (Sez. 6.5.8):  $P=1$ ,  $p=10\%$ , split dinamico; scende sotto la *base* su 5 seed:  $2.04 \times 10^{-1}$  vs  $2.16 \times 10^{-1}$  (12/20).

| Metodo                                                 | base                         | $M=\infty$                                      | def.                                            | $P=1, p=0.1, \text{dyn}$                        | $P=3, p=0.1, \text{dyn}$                        |
|--------------------------------------------------------|------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|
| $\kappa \approx 1.1$ - Dynamic GD                      | $1.2412 \times 10^{-1}$ (29) | $1.2347 \times 10^{-1} \blacktriangle (19)$     | $1.2347 \times 10^{-1} \blacktriangle (19)$     | $1.1704 \times 10^{-1} \blacktriangle (19)$     | $1.1441 \times 10^{-1} \blacktriangle (18)$     |
| $\kappa \approx 1.1$ - BB-CCV                          | $1.2268 \times 10^{-1}$ (29) | $1.2268 \times 10^{-1} \blacktriangledown (8)$  | $1.2268 \times 10^{-1} \blacktriangledown (8)$  | $1.1629 \times 10^{-1} \blacktriangle (26)$     | $1.1629 \times 10^{-1} \blacktriangle (26)$     |
| $\kappa \approx 1.1$ - Newton-CG                       | $3.7346 \times 10^{-1}$ (29) | $1.4142 \times 10^0 \blacktriangledown (0)$     | $2.9780 \times 10^{-1} \blacktriangle (3)$      | $1.7763 \times 10^{-1} \blacktriangle (5)$      | $1.6879 \times 10^{-1} \blacktriangle (6)$      |
| $\kappa \approx 1.1$ - Newton-CG $L_1$                 | $1.1114 \times 10^{-1}$ (29) | $7.7838 \times 10^{-2} \blacktriangle (6)$      | $7.7838 \times 10^{-2} \blacktriangle (6)$      | $8.7102 \times 10^{-2} \blacktriangle (13)$     | $8.7102 \times 10^{-2} \blacktriangle (13)$     |
| $\kappa \approx 20$ - Dynamic GD                       | $5.6422 \times 10^{-2}$ (29) | $6.1835 \times 10^{-2} \blacktriangledown (1)$  | $6.1835 \times 10^{-2} \blacktriangledown (1)$  | $1.5675 \times 10^{-1} \blacktriangledown (27)$ | $1.2093 \times 10^{-2} \blacktriangle (8)$      |
| $\kappa \approx 20$ - BB-CCV                           | $1.7173 \times 10^{-3}$ (29) | $1.7173 \times 10^{-3} \blacktriangle (10)$     | $1.7173 \times 10^{-3} \blacktriangle (13)$     | $3.7758 \times 10^{-2} \blacktriangledown (26)$ | $1.6199 \times 10^{-2} \blacktriangledown (20)$ |
| $\kappa \approx 20$ - Newton-CG                        | $1.7217 \times 10^{-1}$ (29) | $6.4637 \times 10^{-1} \blacktriangledown (0)$  | $3.6965 \times 10^{-1} \blacktriangledown (6)$  | $1.8698 \times 10^{-1} \blacktriangledown (11)$ | $4.0726 \times 10^{-1} \blacktriangledown (5)$  |
| $\kappa \approx 20$ - Newton-CG $L_1$                  | $4.8708 \times 10^{-1}$ (29) | $5.9759 \times 10^{-1} \blacktriangledown (4)$  | $5.9759 \times 10^{-1} \blacktriangledown (4)$  | $6.6464 \times 10^{-1} \blacktriangledown (7)$  | $6.7171 \times 10^{-1} \blacktriangledown (5)$  |
| $\kappa \approx 100$ - Dynamic GD                      | $4.5401 \times 10^{-1}$ (29) | $4.7131 \times 10^{-1} \blacktriangledown (26)$ | $4.7131 \times 10^{-1} \blacktriangledown (26)$ | $5.4144 \times 10^{-1} \blacktriangledown (27)$ | $4.0066 \times 10^{-1} \blacktriangle (26)$     |
| $\kappa \approx 100$ - BB-CCV                          | $1.7173 \times 10^{-3}$ (29) | $1.4573 \times 10^{-3} \blacktriangle (26)$     | $1.4573 \times 10^{-3} \blacktriangle (26)$     | $3.4910 \times 10^{-3} \blacktriangledown (27)$ | $2.2569 \times 10^{-1} \blacktriangledown (25)$ |
| $\kappa \approx 100$ - Newton-CG                       | $1.8251 \times 10^{-1}$ (29) | $1.4142 \times 10^0 \blacktriangledown (0)$     | $8.2559 \times 10^{-1} \blacktriangledown (8)$  | $1.9718 \times 10^{-1} \blacktriangledown (12)$ | $3.9508 \times 10^{-1} \blacktriangledown (5)$  |
| $\kappa \approx 100$ - Newton-CG $L_1$                 | $8.6698 \times 10^{-1}$ (29) | $9.7009 \times 10^{-1} \blacktriangledown (3)$  | $9.6923 \times 10^{-1} \blacktriangledown (4)$  | $9.6870 \times 10^{-1} \blacktriangledown (9)$  | $9.6870 \times 10^{-1} \blacktriangledown (4)$  |
| incrociato ( $\kappa \approx 1.67$ ) - Dynamic GD      | $1.8709 \times 10^{-2}$ (29) | $1.7261 \times 10^{-2} \blacktriangle (16)$     | $1.7261 \times 10^{-2} \blacktriangle (16)$     | $6.1465 \times 10^{-3} \blacktriangle (16)$     | $6.2449 \times 10^{-3} \blacktriangle (15)$     |
| incrociato ( $\kappa \approx 1.67$ ) - BB-CCV          | $9.5580 \times 10^{-3}$ (29) | $9.5580 \times 10^{-3} \blacktriangle (19)$     | $9.5580 \times 10^{-3} \blacktriangle (21)$     | $5.3768 \times 10^{-4} \blacktriangle (27)$     | $5.3768 \times 10^{-4} \blacktriangle (27)$     |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG       | $2.8717 \times 10^{-1}$ (29) | $1.4142 \times 10^0 \blacktriangledown (0)$     | $7.8791 \times 10^{-2} \blacktriangledown (5)$  | $9.2824 \times 10^{-2} \blacktriangledown (11)$ | $1.7781 \times 10^{-1} \blacktriangle (8)$      |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG $L_1$ | $1.0628 \times 10^{-2}$ (29) | $2.2289 \times 10^{-2} \blacktriangledown (11)$ | $2.2289 \times 10^{-2} \blacktriangledown (11)$ | $2.7610 \times 10^{-2} \blacktriangledown (17)$ | $2.7754 \times 10^{-2} \blacktriangledown (14)$ |
| Funzione di Rosenbrock ( $c=100$ ) - Dynamic GD        | $5.3698 \times 10^{-3}$ (29) | $5.3845 \times 10^{-3} \blacktriangledown (22)$ | $5.3845 \times 10^{-3} \blacktriangledown (22)$ | $1.4065 \times 10^{-2} \blacktriangledown (24)$ | $4.7963 \times 10^{-3} \blacktriangle (23)$     |
| Funzione di Rosenbrock ( $c=100$ ) - BB-CCV            | $5.3653 \times 10^{-3}$ (29) | $5.3653 \times 10^{-3} \blacktriangledown (26)$ | $5.3653 \times 10^{-3} \blacktriangledown (26)$ | $5.0034 \times 10^{-3} \blacktriangledown (27)$ | $5.0034 \times 10^{-3} \blacktriangledown (27)$ |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG         | $4.2039 \times 10^{-1}$ (29) | $5.6994 \times 10^{-1} \blacktriangledown (2)$  | $4.8797 \times 10^{-1} \blacktriangledown (3)$  | $7.2464 \times 10^{-2} \blacktriangle (10)$     | $9.7980 \times 10^{-2} \blacktriangle (7)$      |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG $L_1$   | $2.0116 \times 10^{-1}$ (29) | $1.2973 \times 10^{-1} \blacktriangle (2)$      | $1.2973 \times 10^{-1} \blacktriangle (2)$      | $1.1021 \times 10^{-1} \blacktriangle (3)$      | $1.0884 \times 10^{-1} \blacktriangle (3)$      |
| Media (20 casi)                                        | $1.9562 \times 10^{-1}$      | $4.0383 \times 10^{-1} \blacktriangledown$      | $2.3383 \times 10^{-1} \blacktriangledown$      | $1.7919 \times 10^{-1} \blacktriangle$          | $2.0065 \times 10^{-1} \blacktriangledown$      |

Dalla tesi — Sez. 6.5.6 e Tab. 6.3 (seed 42): tra parentesi il numero di ricampionamenti su 30 iterazioni. *def.* = valori di default ( $P=3$ ,  $p=20\%$ , split fisso).

## Riuso per discesa della loss sul batch: come funziona

- Stesso obiettivo dello stop adattivo — capire quando il batch è “esaurito” — ma **senza riservare dati**: si osserva solo la loss sul mini-batch corrente, campionato dall’*intero* dataset (CCV con tetto  $N$ ).
- Le line search garantiscono già  $J_{\text{batch}}$  decrescente a ogni passo accettato: il criterio non scatta se la loss *cresce*, ma se *decresce troppo poco* (riduzione relativa sotto soglia), tipico in prossimità del minimo campionario del batch.

**Pseudocodice** (iterazione esterna  $k$ , batch in riuso):

1.  $S_k \leftarrow$  mini-batch dall’intero dataset;  $g_k = \nabla J_{S_k}(w_k)$ ; aggiorna  $w_{k+1}$  con la line search.
2. Ogni  $f$  iterazioni valuta  $J_{\text{batch}}(w_k)$  e la confronta con la valutazione precedente *sullo stesso batch* (il confronto riparte da zero dopo ogni ricampionamento).
3. *Progresso* se  $J_{\text{batch}}(w_{k-1}) - J_{\text{batch}}(w_k) \geq \tau |J_{\text{batch}}(w_{k-1})| + \epsilon_{\text{abs}}$ .
4. Se per  $P$  valutazioni consecutive *non* c’è progresso  $\Rightarrow$  **ricampiona** (batch “esaurito”);  $\tau$  più alta (es.  $10^{-3}$ ) dichiara il batch esaurito prima.

Dalla tesi — Sez. 6.5.7. Per Newton-CG  $L_1$  si monitora  $F_{\text{batch}} = J_{\text{batch}} + \nu \|w\|_1$  sul batch.

# In alternativa: riuso per discesa della loss sul batch (Tab. 6.5)

- Alternativa *senza* validation set: si osserva la sola loss sul batch corrente (“esaurito” quando la riduzione relativa è sotto soglia).
- Per i metodi di Newton evita il collasso di  $M=\infty$  ( $\kappa \approx 20$ : da  $1.28$  a  $2.48 \times 10^{-1}$ ); per Dynamic GD/BB-CCV quasi come  $M=\infty$ .

| Metodo                                                 | base                          | $M=\infty$                                      | def.                                            | $P=1, \tau=10^{-3}, f=1$                        | $P=1, \tau=10^{-5}, f=1$                        |
|--------------------------------------------------------|-------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|-------------------------------------------------|
| $\kappa \approx 1.1$ - Dynamic GD                      | $1.1895 \times 10^{-1}$ (29)  | $1.1739 \times 10^{-1} \blacktriangle$ (18)     | $1.1739 \times 10^{-1} \blacktriangle$ (18)     | $1.1739 \times 10^{-1} \blacktriangle$ (18)     | $1.1739 \times 10^{-1} \blacktriangle$ (18)     |
| $\kappa \approx 1.1$ - BB-CCV                          | $1.1662 \times 10^{-1}$ (11)  | $1.1662 \times 10^{-1} \blacktriangle$ (5)      | $1.1662 \times 10^{-1} \blacktriangle$ (5)      | $1.1662 \times 10^{-1} \blacktriangle$ (5)      | $1.1662 \times 10^{-1} \blacktriangle$ (5)      |
| $\kappa \approx 1.1$ - Newton-CG                       | $3.0560 \times 10^{-1}$ (29)  | $2.2230 \times 10^{-1} \blacktriangle$ (2)      | $1.6534 \times 10^{-1} \blacktriangle$ (4)      | $1.6534 \times 10^{-1} \blacktriangle$ (4)      | $1.6534 \times 10^{-1} \blacktriangle$ (4)      |
| $\kappa \approx 1.1$ - Newton-CG $L_1$                 | $1.0028 \times 10^{-1}$ (29)  | $8.1750 \times 10^{-2} \blacktriangle$ (8)      | $8.1750 \times 10^{-2} \blacktriangle$ (8)      | $8.1750 \times 10^{-2} \blacktriangle$ (8)      | $8.1750 \times 10^{-2} \blacktriangle$ (8)      |
| $\kappa \approx 20$ - Dynamic GD                       | $4.9050 \times 10^{-2}$ (29)  | $1.0103 \times 10^{-1} \blacktriangledown$ (1)  | $1.0103 \times 10^{-1} \blacktriangledown$ (1)  | $4.8866 \times 10^{-2} \blacktriangle$ (14)     | $1.0103 \times 10^{-1} \blacktriangledown$ (1)  |
| $\kappa \approx 20$ - BB-CCV                           | $9.7195 \times 10^{-5}$ (29)  | $9.5364 \times 10^{-10} \blacktriangle$ (14)    | $9.5364 \times 10^{-10} \blacktriangle$ (14)    | $9.5364 \times 10^{-10} \blacktriangle$ (14)    | $9.5364 \times 10^{-10} \blacktriangle$ (14)    |
| $\kappa \approx 20$ - Newton-CG                        | $1.6683 \times 10^{-1}$ (29)  | $1.2807 \times 10^0 \blacktriangledown$ (0)     | $2.4844 \times 10^{-1} \blacktriangledown$ (8)  | $2.4844 \times 10^{-1} \blacktriangledown$ (8)  | $2.4844 \times 10^{-1} \blacktriangledown$ (8)  |
| $\kappa \approx 20$ - Newton-CG $L_1$                  | $5.4573 \times 10^{-1}$ (29)  | $7.6221 \times 10^{-1} \blacktriangledown$ (5)  | $7.6221 \times 10^{-1} \blacktriangledown$ (5)  | $7.6221 \times 10^{-1} \blacktriangledown$ (5)  | $7.6221 \times 10^{-1} \blacktriangledown$ (5)  |
| $\kappa \approx 100$ - Dynamic GD                      | $4.6668 \times 10^{-1}$ (29)  | $3.7060 \times 10^{-1} \blacktriangle$ (27)     | $3.7060 \times 10^{-1} \blacktriangle$ (27)     | $3.7060 \times 10^{-1} \blacktriangle$ (27)     | $3.7060 \times 10^{-1} \blacktriangle$ (27)     |
| $\kappa \approx 100$ - BB-CCV                          | $1.0991 \times 10^{-14}$ (10) | $7.4538 \times 10^{-3} \blacktriangledown$ (24) | $7.4538 \times 10^{-3} \blacktriangledown$ (24) | $7.4538 \times 10^{-3} \blacktriangledown$ (24) | $7.4538 \times 10^{-3} \blacktriangledown$ (24) |
| $\kappa \approx 100$ - Newton-CG                       | $2.3041 \times 10^{-1}$ (29)  | $1.2807 \times 10^0 \blacktriangledown$ (0)     | $4.2582 \times 10^{-1} \blacktriangledown$ (11) | $4.2582 \times 10^{-1} \blacktriangledown$ (11) | $4.2582 \times 10^{-1} \blacktriangledown$ (11) |
| $\kappa \approx 100$ - Newton-CG $L_1$                 | $8.6562 \times 10^{-1}$ (29)  | $9.7065 \times 10^{-1} \blacktriangledown$ (5)  | $9.7065 \times 10^{-1} \blacktriangledown$ (5)  | $9.7065 \times 10^{-1} \blacktriangledown$ (5)  | $9.7065 \times 10^{-1} \blacktriangledown$ (5)  |
| incrociato ( $\kappa \approx 1.67$ ) - Dynamic GD      | $9.4161 \times 10^{-3}$ (29)  | $5.5315 \times 10^{-3} \blacktriangle$ (16)     | $5.5315 \times 10^{-3} \blacktriangle$ (16)     | $5.5315 \times 10^{-3} \blacktriangle$ (16)     | $5.5315 \times 10^{-3} \blacktriangle$ (16)     |
| incrociato ( $\kappa \approx 1.67$ ) - BB-CCV          | $6.1644 \times 10^{-4}$ (15)  | $6.1689 \times 10^{-4} \blacktriangledown$ (9)  | $6.1689 \times 10^{-4} \blacktriangledown$ (9)  | $6.1689 \times 10^{-4} \blacktriangledown$ (9)  | $6.1689 \times 10^{-4} \blacktriangledown$ (9)  |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG       | $2.6381 \times 10^{-1}$ (29)  | $4.1026 \times 10^{-2} \blacktriangle$ (5)      | $4.1026 \times 10^{-2} \blacktriangle$ (5)      | $4.1026 \times 10^{-2} \blacktriangle$ (5)      | $4.1026 \times 10^{-2} \blacktriangle$ (5)      |
| incrociato ( $\kappa \approx 1.67$ ) - Newton-CG $L_1$ | $2.5154 \times 10^{-2}$ (29)  | $3.8938 \times 10^{-2} \blacktriangledown$ (10) | $3.8938 \times 10^{-2} \blacktriangledown$ (10) | $3.8938 \times 10^{-2} \blacktriangledown$ (10) | $3.8938 \times 10^{-2} \blacktriangledown$ (10) |
| Funzione di Rosenbrock ( $c=100$ ) - Dynamic GD        | $6.1354 \times 10^{-4}$ (29)  | $2.5435 \times 10^0 \blacktriangledown$ (25)    | $2.5435 \times 10^0 \blacktriangledown$ (25)    | $2.5435 \times 10^0 \blacktriangledown$ (25)    | $2.5435 \times 10^0 \blacktriangledown$ (25)    |
| Funzione di Rosenbrock ( $c=100$ ) - BB-CCV            | $1.0645 \times 10^{-1}$ (29)  | $1.6648 \times 10^{-1} \blacktriangledown$ (10) | $1.6648 \times 10^{-1} \blacktriangledown$ (10) | $1.6648 \times 10^{-1} \blacktriangledown$ (10) | $1.6648 \times 10^{-1} \blacktriangledown$ (10) |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG         | $9.3953 \times 10^{-1}$ (29)  | $1.0605 \times 10^0 \blacktriangledown$ (0)     | $1.0247 \times 10^0 \blacktriangledown$ (4)     | $1.0247 \times 10^0 \blacktriangledown$ (4)     | $1.0247 \times 10^0 \blacktriangledown$ (4)     |
| Funzione di Rosenbrock ( $c=100$ ) - Newton-CG $L_1$   | $2.0207 \times 10^{-1}$ (29)  | $3.8199 \times 10^{-1} \blacktriangledown$ (0)  | $3.8199 \times 10^{-1} \blacktriangledown$ (0)  | $3.8199 \times 10^{-1} \blacktriangledown$ (0)  | $3.8199 \times 10^{-1} \blacktriangledown$ (0)  |

Dalla tesi — Sez. 6.5.7 e Tab. 6.5 (seed 42): tra parentesi il numero di ricampionamenti su 30 iterazioni. *def.* = valori di default ( $\tau=10^{-4}$ ,  $P=1$ ,  $f=1$ ).

## Confronto finale su 5 seed: base, $M=\infty$ , validation e discesa (Tab. 6.7)

- $\bar{e}_{30}$  = media su 20 combinazioni problema×algoritmo e 5 seed; *vitt.* = caselle (su 20) in cui la strategia batte la *base*.
- $M=\infty$  è la peggiore in media ( $3.73 \times 10^{-1}$ ); validation calib.  $2.04 \times 10^{-1}$  (12/20), discesa calib.  $2.38 \times 10^{-1}$  (11/20).
- La discesa usa l'intero dataset; la validation tiene fuori il 10% dei dati.

| Strategia                                      | $\bar{e}_{30}$          | <i>vitt.</i> | $\kappa \approx 1.1$    | $\kappa \approx 20$     | $\kappa \approx 100$    | incr. ( $\kappa \approx 1.67$ ) | Funzione di Rosenbrock ( $c=100$ ) |
|------------------------------------------------|-------------------------|--------------|-------------------------|-------------------------|-------------------------|---------------------------------|------------------------------------|
| <i>base</i>                                    | $2.0909 \times 10^{-1}$ | 0/20         | $1.7930 \times 10^{-1}$ | $1.7595 \times 10^{-1}$ | $4.1162 \times 10^{-1}$ | $8.1419 \times 10^{-2}$         | $1.9719 \times 10^{-1}$            |
| $M=\infty$                                     | $3.7299 \times 10^{-1}$ | 8/20         | $1.9159 \times 10^{-1}$ | $5.0123 \times 10^{-1}$ | $6.6432 \times 10^{-1}$ | $9.8773 \times 10^{-2}$         | $4.0901 \times 10^{-1}$            |
| Validation calib. ( $P=1, p=0.1, \text{dyn}$ ) | $2.0375 \times 10^{-1}$ | 12/20        | $1.4045 \times 10^{-1}$ | $2.1065 \times 10^{-1}$ | $4.5437 \times 10^{-1}$ | $3.9223 \times 10^{-2}$         | $1.7407 \times 10^{-1}$            |
| Discesa calib. ( $\tau=10^{-3}, P=1, f=1$ )    | $2.3817 \times 10^{-1}$ | 11/20        | $1.1855 \times 10^{-1}$ | $2.3416 \times 10^{-1}$ | $4.2013 \times 10^{-1}$ | $3.4413 \times 10^{-2}$         | $3.8362 \times 10^{-1}$            |

Dalla tesi — Sez. 6.5.6–6.5.7 e Tab. 6.7: confronto dei due criteri automatici su 5 seed. “*calib.*” = parametri del criterio scelti con una ricerca a griglia (Sez. 6.5.8), non i default: validation  $P=1$ ,  $p=10\%$ , split dinamico (default  $P=3$ ,  $p=20\%$ , split fisso); discesa  $\tau=10^{-3}$ ,  $P=1$ ,  $f=1$  (default  $\tau=10^{-4}$ ).

# Conclusioni

- Quadro completo: dalla varianza del gradiente stocastico (SGD) e dai metodi a varianza ridotta (SVRG/SAGA) si arriva a una regola *dinamica e basata sui dati* per la dimensione del campione.
- Arresto adattivo del CG per Newton-CG: criterio automatico basato sulla varianza dei prodotti Hessiana–vettore.
- Implementazione Python dei quattro algoritmi + app web interattiva per ripetere gli esperimenti.
- Esperimenti sui problemi quadratici sintetici (da  $\kappa \approx 1.1$  a  $\kappa \approx 100$ ) e sulla Funzione di Rosenbrock; analizzato anche il riuso del mini-batch con stop adattivo.

## Limiti e lavoro futuro

- Limite attuale: esperimenti solo su problemi sintetici (quadratici centrati,  $\kappa \approx 1.1, 20, 100$ , termine incrociato, Funzione di Rosenbrock).
- Limite attuale: nessun confronto sistematico con SVRG/SAGA sulle stesse batterie di test.
- Futuro: rapporto di sottocampionamento  $R$  dell'Hessiana dinamico (oggi  $R$  è fisso: si adatta solo  $n_k$  del gradiente).
- Futuro: validazione su dataset reali e problemi non convessi, confronto sistematico con SVRG/SAGA su larga scala.